Shri Vile Parle Kelavani Mandal'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai

Department of Information Technology

COURSE CODE: DJS23ICPC501 **DATE: 08/10/2026**
COURSE NAME: Computer Networks **CLASS: T. Y. B.Tech**
BATCH: I3-1

Experiment No. 9

CO/LO: Implement Transport Layer Protocols.

Aim: To implement sockets using UDP

EXERCISE

```
Server: import
socket

# Initialize UDP socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

SERVER_IP = "0.0.0.0"  # Listens on all network interfaces SERVER_PORT
= 5010
server_socket.bind((SERVER_IP, SERVER_PORT))

# Dictionary to map client addresses to their chosen usernames -> {(IP, Port): "Username"}
clients = {}

print("=== UDP GROUP CHAT SERVER ===") print(f"Server
is running on port {SERVER_PORT}...")

while True:
  try:
      data, client_address = server_socket.recvfrom(1024)
  message = data.decode()
      # Handle a brand new connection
if client_address not in clients:
```






```
        # The very first message a client sends will be treated as their username
username = message
        clients[client_address] = username
        print(f"[JOIN] {username} {client_address} connected.")

        # Notify EVERYONE ELSE that a new user joined
        join_alert = f"\n*** {username} has joined the chat room! ***".encode()
for client in clients:
            if client != client_address: # <-- EXCLUDES THE SENDER
server_socket.sendto(join_alert, client)            continue

    # Get the username of the sender
sender_username = clients[client_address]

    # Handle client leaving
if message.lower() == "bye":
        print(f"[LEAVE] {sender_username} {client_address} disconnected.")

        # Notify EVERYONE ELSE that this user left
        leave_alert = f"\n*** {sender_username} has left the chat room. ***".encode()

        # Remove from active list *after* extracting username but *before* broadcasting
del clients[client_address]

        for client in clients:
            server_socket.sendto(leave_alert, client)
continue

    # Print to server console for monitoring logs
    print(f"[{sender_username}]: {message}")

    # Broadcast message to ALL active clients EXCEPT the sender
  formatted_message = f"\n[{sender_username}]: {message}".encode()
  for client in clients:
```






```
        if client != client_address:  # <-- EXCLUDES THE SENDER
  server_socket.sendto(formatted_message, client)

    except Exception as e:
        print(f"Server Error: {e}")
        break

server_socket.close()
```

Client: import
socket

```
import threading
import sys

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Change this to your server's actual IP address if testing across computers
SERVER_IP = "10.120.64.34"
SERVER_PORT = 5010
SERVER_ADDRESS = (SERVER_IP, SERVER_PORT)

print("=== UDP GROUP CHAT CLIENT ===")
username = input("Enter your username to join: ").strip()

# Send username first so the server registers you
client_socket.sendto(username.encode(), SERVER_ADDRESS)
print(f"Welcome to the room, {username}! Type 'bye' to quit.\n")

# Background thread: Continually listens for messages from the server def
receive_messages():
    while True:
try:
```






```
        data, _ = client_socket.recvfrom(1024)
# Print the incoming chat message
        print(data.decode())
        # Clean up the command line prompt so you can keep typing
  seamlessly          print("You: ", end="", flush=True)      except Exception:
        break
receive_thread = threading.Thread(target=receive_messages) receive_thread.daemon
= True
receive_thread.start()

# Main thread loop: Handles your keyboard typing input
while True:     try:
      message = input("You: ")

      # Send message to the central server
      client_socket.sendto(message.encode(), SERVER_ADDRESS)

      if message.lower() == "bye":
print("Leaving chat room...")
        break     except (KeyboardInterrupt,
SystemExit):
      client_socket.sendto("bye".encode(), SERVER_ADDRESS)
break

client_socket.close() sys.exit()
```

Shri Vile Parle Kelavani Mandal'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Empowered Autonomous) Affiliated to the University of Mumbai

Department of Information Technology

```
PS E:\I075> python server.py
=== UDP GROUP CHAT SERVER ===
Server is running on port 5010...
[JOIN] akg ('10.120.64.33', 58702) connected.
[akg]: hi i am akg
[JOIN] preet_dalal ('10.120.64.35', 50187) connected.
[akg]: who all are there in the room ?
[JOIN] Hitansh ('10.120.64.34', 54210) connected.
[JOIN] preet_dalal ('10.120.64.35', 49490) connected.
[Hitansh]: Hi Preet
[preet_dalal]: Hello Hitansh
[preet_dalal]: How are you?
[akg]: oday is a very sunny day
[Hitansh]: Yes
[Hitansh]: Today is Tuesday
[akg]: toady is wednesday
[preet_dalal]: Tuesday timings are 10 to 6
[Hitansh]: yes
[akg]: tuesday is boring
[Hitansh]: Tuesday is Not Boring
[akg]: thursday is GOAT
[LEAVE] akg ('10.120.64.33', 58702) disconnected.
[preet_dalal]: Friday is my favourite day
[LEAVE] Hitansh ('10.120.64.34', 54210) disconnected.
[LEAVE] preet_dalal ('10.120.64.35', 49490) disconnected.
[JOIN] aryan gada ('10.120.64.33', 50372) connected.
[aryan gada]: hello
[LEAVE] aryan gada ('10.120.64.33', 50372) disconnected.
```

```
PS E:\I075> python client.py
=== UDP GROUP CHAT CLIENT ===
Enter your username to join: Hitansh
Welcome to the room, Hitansh! Type 'bye' to quit.

You:
*** preet_dalal has joined the chat room! ***
You: Hi Preet
You:
[preet_dalal]: Hello Hitansh
You:
[preet_dalal]: How are you?
You:
[akg]: oday is a very sunny day
You: Yes
You: Today is Tuesday
You:
[akg]: toady is wednesday
You:
[preet_dalal]: Tuesday timings are 10 to 6
You: yes
You:
[akg]: tuesday is boring
You: Tuesday is Not Boring
You:
[akg]: thursday is GOAT
You:
*** akg has left the chat room. ***
You:
[preet_dalal]: Friday is my favourite day
You: bye
Leaving chat_room...
```

Department of Information Technology

Conclusion:

The experiment successfully demonstrated the implementation of a **UDP-based group chat application using Python sockets**. The server used a UDP socket to receive datagrams from multiple clients and maintained their addresses and usernames for message forwarding. Clients were able to join the chat, exchange messages with other clients, and leave the chat using the bye command. A separate receiving thread allowed clients to **send and receive messages simultaneously**, enabling two-way communication. The experiment also demonstrated the connectionless nature of UDP, where messages are transmitted using independent datagrams without establishing a dedicated connection.

Reference:

[1] www.geeksforgeeks.org